GMSK Modem Project

This repository contains a staged implementation of a GMSK communication modem, starting from a MATLAB floating-point communication loop and extending toward hardware implementation using Xilinx Vitis HLS and Vivado.

The project is organized into four main parts:

```
gmskModem/
|-- gmsk-project-phase01/
|-- gmsk-project-phase02/
|-- gmsk-snr-vs-ber/
`-- HLS-implementation/
```

Project Summary

The implemented communication chain reads payload data from a CSV file, converts it into packets, applies convolutional coding, modulates the packet using GMSK, passes the waveform through a channel model, demodulates the received signal, decodes the payload, and writes the recovered data back to a CSV file.

The main development path is:

1. Build and verify the complete MATLAB GMSK packet modem under AWGN.
2. Replace library-dependent coding blocks with custom convolutional encoder and soft-decision Viterbi decoder implementations.
3. Add more realistic GSM hilly-terrain channel behavior and LMS equalization.
4. Sweep BER against SNR for ideal and realistic channel cases.
5. Implement the convolutional encoder and Viterbi decoder in HLS.
6. Generate RTL and measure the throughput of an encoder-decoder loop on a Zybo Z7-10 FPGA board.

Communication Pipeline

The high-level modem pipeline is:

```
CSV payload
  -> AXI-stream CSV reader
  -> packet builder
  -> rate-1/2 convolutional encoder
  -> GMSK modulator
  -> channel model
  -> GMSK demodulator
  -> preamble detection
  -> payload extraction
  -> soft-decision Viterbi decoder
  -> recovered payload bytes
  -> AXI-stream CSV writer
```

The waveform uses a burst-based packet structure:

```
48 preamble bits
+ 544 encoded payload bits
+ 32 training bits
= 624 burst bits
```

The payload is 34 bytes, or 272 information bits. After rate-1/2 convolutional encoding, the payload becomes 544 encoded bits. With the default sample rate of 4 samples per symbol, one burst contains 2496 complex samples.

The packet layout is:

```
[Preamble |
 Payload chunk 1 | Training chunk 1 |
 Payload chunk 2 | Training chunk 2 |
 Payload chunk 3 | Training chunk 3 |
 Payload chunk 4 | Training chunk 4]
```

Phase 01: MATLAB AWGN Communication Loop

Folder:

```
gmsk-project-phase01/
```

Phase 01 contains the initial MATLAB implementation of the complete packet communication loop. It focuses on the ideal AWGN-channel version of the modem and verifies that data can be recovered from the full transmit-receive chain.

Main features:

- Reads transmit bytes from AXI-stream style CSV files.
- Builds packet bursts containing preamble, payload, and training fields.
- Applies rate-1/2 convolutional encoding.
- Performs Gaussian filtering and GMSK modulation.
- Sends the signal through an AWGN channel.
- Demodulates the received GMSK signal into soft metrics.
- Detects the preamble and extracts the encoded payload.
- Uses MATLAB/library-style soft-decision Viterbi decoding.
- Converts decoded bits back to bytes and writes output CSV files.
- Includes single-packet and multi-packet test flows.

Important files:

File	Purpose
`test_complete_system.m`	Main single-packet end-to-end AWGN test.
`test_multipacket_system.m`	Multi-packet transmit and receive test.
`read_axi_stream_csv.m`	Reads AXI-stream style CSV input.
`write_axi_stream_csv.m`	Writes recovered data to AXI-stream style CSV.
`build_packet.m`	Builds one burst from payload bytes.
`build_multipacket.m`	Builds a continuous multi-packet stream.
`convEncoder.m`	Convolutional encoder block.
`ViterbiDecoder.m`	Soft-decision Viterbi decoder block.
`GaussianFilter.m`	Gaussian pulse-shaping filter.
`GmskMapper.m`	GMSK modulator.
`GmskDemapper.m`	GMSK demapper.
`RxDemodulator.m`	Receiver demodulation block.
`detect_preamble.m`	Correlation-based burst detection.
`extract_payload.m`	Extracts payload fields from detected burst.

To run the main Phase 01 test:

```
cd gmsk-project-phase01
test_complete_system
```

To run the multi-packet test:

```
cd gmsk-project-phase01
test_multipacket_system
```

More details are available in:

```
gmsk-project-phase01/SYSTEM_README.md
gmsk-project-phase01/Report/Report.pdf
```

Phase 02: Custom Coding Blocks, GSM Channel, and LMS Equalization

Folder:

```
gmsk-project-phase02/
```

Phase 02 extends the Phase 01 modem. The transmit-receive pipeline is similar, but the convolutional encoder and Viterbi decoder are implemented as custom MATLAB modules. This phase also adds realistic channel behavior and receiver equalization.

Main additions:

- Custom convolutional encoder implementation.
- Custom soft-decision Viterbi decoder implementation.
- GSM hilly-terrain channel model using `gsmHTx12c1`.
- AWGN added after fading.
- Coarse burst detection using the known preamble.
- Complex LMS/NLMS equalization using known preamble and training samples.
- Channel-estimation/equalizer support utilities.
- Payload byte comparison utility for transmit-vs-receive checking.

Main execution flow:

```
tx_data.csv
  -> tx_chain.m
  -> channel_model.m
  -> rx_chain.m
  -> rx_data.csv
  -> compare_payload_bytes.m
```

Important files:

File	Purpose
`main_system.m`	Runs the complete Phase 02 TX, channel, RX, and comparison flow.

File	Purpose
tx_chain.m	Reads CSV payload, builds the burst, and generates the TX signal.
channel_model.m	Applies GSM HT fading and AWGN.
rx_chain.m	Detects, equalizes, demodulates, decodes, and writes recovered bytes.
convEncoder.m	Custom rate-1/2 convolutional encoder.
viterbi_soft_decoder.m	Custom soft-decision Viterbi decoder.
lms_equalizer_complex.m	Complex NLMS equalizer.
preamble_sample_indices.m	Converts known burst regions into training sample indices.
gmsk_modulate_burst_ref.m	Generates the reference GMSK burst for equalizer training.
demod_aligned_burst.m	Demodulates already aligned/equalized bursts.
sweep_equalizer_params.m	Sweeps LMS equalizer parameters.

To run Phase 02:

```
cd gmsk-project-phase02
main_system
```

The default SNR is set in main_system.m:

```
SNR_dB = 30;
```

To tune the equalizer:

```
cd gmsk-project-phase02
results = sweep_equalizer_params('tx_data.csv', 30);
```

More details are available in:

```
gmsk-project-phase02/README.md
gmsk-project-phase02/README.pdf
```

BER vs SNR Experiments

Folder:

```
gmsk-snr-vs-ber/
```

This folder contains BER-vs-SNR sweeps for the GMSK packet communication loop. The goal of these simulations is to estimate the SNR at which the BER reaches the target value of:

```
BER = 1e-5
```

Three channel cases are included.

1. Ideal AWGN Channel

Folder:

```
gmsk-snr-vs-ber/ideal_curve/
```

This is the ideal reference case. The signal is transmitted through AWGN only, without multipath fading or equalization.

Main script:

```
cd gmsk-snr-vs-ber/ideal_curve
ideal_snr_vs_ber
```

The script sweeps SNR, measures BER, plots the curve, and marks the target BER level of `1e-5`.

2. GSM HT Channel Without Equalization

Folder:

```
gmsk-snr-vs-ber/gsmHTx12c1_channel_without_equalization/
```

This case adds the realistic GSM hilly-terrain channel model `gsmHTx12c1` but does not use LMS equalization. It shows the BER degradation caused by the real channel conditions.

Main script:

```
cd gmsk-snr-vs-ber/gsmHTx12c1_channel_without_equalization
rayleigh_ber_vs_snr
```

3. GSM HT Channel With LMS Equalization

Folder:

```
gmsk-snr-vs-ber/gsmHTx12c1_channel_with_lms_equalization/
```

This case uses the GSM HT channel and adds LMS equalization in the receiver. It is used to measure the improvement from equalization and to estimate the SNR required to reach `1e-5` BER under the more realistic channel model.

Main script:

```
cd gmsk-snr-vs-ber/gsmHTx12c1_channel_with_lms_equalization
plot_ber_vs_snr
```

Each BER experiment folder includes a generated `Figure_1.png` plot and local README files with case-specific notes.

HLS and FPGA Implementation

Folder:

```
HLS-implementation/
```

This folder contains the hardware-oriented implementation work. The full GMSK communication pipeline was investigated for HDL implementation, but the completed HLS blocks in this repository are the convolutional encoder and the Viterbi decoder.

Tool versions and target board:

Item	Value
HLS tool	Xilinx Vitis HLS 2025.2
FPGA flow	Vivado 2025.2
Board	Digilent Zybo Z7-10
Throughput design clock parameter	125 MHz

HLS Convolutional Encoder

Folder:

```
HLS-implementation/convolutional-encoder/
```

This block implements a rate-1/2 convolutional encoder for the same K=7, polynomial `[171 133]` code used in the MATLAB modem.

Important files:

File	Purpose
`convEncoder_Seq.cpp`	AXI-stream HLS convolutional encoder.
`convEncoder_Seq_tb.cpp`	HLS test bench.
`hls_config.cfg`	Vitis HLS configuration.
`convEncoder/`	Generated HLS project outputs, reports, and RTL.

The encoder accepts one 32-bit input word and emits two 32-bit output words because the code rate is 1/2.

HLS Viterbi Decoder

Folder:

```
HLS-implementation/viterbi-soft-decision-decoder/
```

This block implements a traceback Viterbi decoder for a K=7, rate-1/2 convolutional code. The folder name refers to the soft-decision decoder work, but the checked-in HLS source uses packed 2-bit channel symbols with a Hamming-distance branch metric.

Important files:

File	Purpose
`include/constants.h`	Decoder constants, trellis size, traceback length, and types.
`include/viterbi_decoder.h`	Top-level decoder function declaration.
`include/bmu.h`	Branch metric helper declarations.
`src/viterbi_decoder.cpp`	HLS decoder implementation.
`src/bmu.cpp`	Branch metric unit implementation.
`tb/viterbi_decoder_tb.cpp`	Test bench.

The decoder architecture contains:

- Branch Metric Unit (BMU)
- Add-Compare-Select (ACS) logic over 64 states
- Survivor memory
- Minimum path-metric search
- Traceback unit
- AXI-stream input and output interfaces

FPGA Throughput Measurement

Folder:

```
HLS-implementation/throughput_zybo/
```

The HLS-generated Verilog for the convolutional encoder and Viterbi decoder was connected into a small hardware throughput test loop:

```
lfsr32 -> convolutional encoder RTL -> Viterbi decoder RTL -> throughput counter
```

The input data is generated internally by `lfsr32.sv`, so no processor software or external input data file is required for this measurement.

Important files:

File	Purpose
`throughput_zybo.xpr`	Vivado project.
`throughput_zybo.srcs/sources_1/new/throughput_bench_top.sv`	Top-level throughput bench.
`throughput_zybo.srcs/sources_1/new/lfsr32.sv`	32-bit pseudo-random input source.
`throughput_zybo.srcs/sources_1/new/convEncoder_Seq*.v`	HLS-generated encoder RTL.
`throughput_zybo.srcs/sources_1/new/viterbi_decoder*.v`	HLS-generated decoder RTL.
`throughput_zybo.srcs/constrs_1/new/constraints.xdc`	Board constraints.

Throughput is calculated as:

```
throughput_bits_per_second = (bits_q * clock_frequency_hz) / cycs_q
```

For Mbps:

```
throughput_Mbps = ((bits_q * clock_frequency_hz) / cycs_q) / 1_000_000
```

The design was measured both in simulation and after synthesis/implementation on the Zybo Z7-10 board. In both cases the measured decoded-output throughput was approximately:

```
32-33 Mbps
```

More details are available in:

```
HLS-implementation/throughput_zybo/README.md
HLS-implementation/throughput_zybo/README.pdf
```

CSV Input and Output Format

The MATLAB modem uses AXI-stream style CSV files. A typical CSV contains:

```
TDATA,TKEEP,TLAST
9B553F84,f,0
0789DC7F,f,0
CA04CA04,3,1
```

Fields:

Field	Meaning
`TDATA`	32-bit data word in hexadecimal.
`TKEEP`	Byte-enable mask. `f` means all four bytes are valid.
`TLAST`	End-of-frame marker.

The helper files `read_axi_stream_csv.m` and `write_axi_stream_csv.m` are used throughout the MATLAB phases and BER experiments.

Requirements

MATLAB simulations:

- MATLAB
- Communications Toolbox for `stdchan`, `physconst`, and related channel modeling functions

HLS/FPGA implementation:

- Xilinx Vitis HLS 2025.2
- Xilinx Vivado 2025.2
- Digilent Zybo Z7-10 board for the included hardware throughput project

Suggested Run Order

For a new checkout or review, use this order:

1. Run the ideal Phase 01 single-packet test.

   ```
   cd gmsk-project-phase01
   ```

```
test_complete_system
```

2. Run the Phase 02 realistic-channel system.

```
cd gmsk-project-phase02
main_system
```

3. Run the BER sweep scripts from the three BER experiment folders.

```
cd gmsk-snr-vs-ber/ideal_curve
ideal_snr_vs_ber
```

```
cd gmsk-snr-vs-ber/gsmHTx12c1_channel_without_equalization
rayleigh_ber_vs_snr
```

```
cd gmsk-snr-vs-ber/gsmHTx12c1_channel_with_lms_equalization
plot_ber_vs_snr
```

4. Open the HLS projects or sources under `HLS-implementation/` to inspect the encoder and decoder hardware implementations.

5. Open the Vivado project:

```
HLS-implementation/throughput_zybo/throughput_zybo.xpr
```

   Use the included throughput bench to reproduce the FPGA throughput measurement.

Current Project Status

- MATLAB GMSK packet communication loop is implemented.
- AWGN channel simulation is implemented.
- Custom MATLAB convolutional encoder and soft-decision Viterbi decoder are implemented.
- GSM HT channel model and LMS equalization are implemented.
- BER-vs-SNR experiments are included for AWGN, GSM HT without equalization, and GSM HT with LMS equalization.
- HLS implementation is completed for the convolutional encoder and Viterbi decoder blocks.
- A hardware throughput bench using generated RTL has been implemented and tested on Zybo Z7-10.
- Full hardware implementation of the entire GMSK modulation/channel/ demodulation pipeline is not completed in this repository.